

ENTRY-LEVEL FULL-STACK

Coding Assignment

Logistics Shipment Tracker

Application type: Full-stack CRUD web application

Backend: Java 21 and Spring Boot REST API

Frontend: React and Vite

Database: PostgreSQL

Execution: Docker Compose

Recommended completion time: 6-10 hours

Suggested submission window: 3-5 calendar days

A focused end-to-end assessment covering CRUD development, REST design, validation, testing, containerisation, Git, and technical documentation.

1. Purpose

Build a small end-to-end logistics application that allows a user to create, view, update, delete, and filter shipment records.

This assignment is intended to assess basic knowledge of:

- Java and Spring Boot
- REST API design
- React UI development
- PostgreSQL and data persistence
- Input validation and error handling
- Unit testing
- Docker and Docker Compose
- Git usage and technical documentation

The application does **not** require authentication, cloud deployment, microservices, messaging, maps, or external APIs.

2. Business Scenario

A small logistics team needs a simple web application to record shipments and monitor their current delivery status.

Each shipment travels from an origin to a destination using a carrier. The team must be able to update its status as it moves through the delivery process.

3. Required Feature

Shipment Management

The user must be able to:

1. Create a shipment.
2. View all shipments.
3. View the details of one shipment.
4. Edit an existing shipment.
5. Delete a shipment after confirmation.
6. Filter shipments by status.
7. See a clear message when no shipments match the selected filter.

These are the complete required business features. Keep the solution simple and focused.

4. Shipment Data Model

Field	Type	Rules
id	Long	Database-generated primary key
trackingNumber	String	Required, unique, 5-30 characters

Field	Type	Rules
senderName	String	Required, 2-100 characters
receiverName	String	Required, 2-100 characters
origin	String	Required, 2-100 characters
destination	String	Required, 2-100 characters
carrier	String	Required, 2-100 characters
status	Enum	Required; one of the allowed values below
expectedDeliveryDate	Date	Required; must not be earlier than the shipment's creation date
createdAt	Date/time	Set automatically by the backend; not editable
updatedAt	Date/time	Updated automatically by the backend

Allowed shipment statuses

- CREATED
- PICKED_UP
- IN_TRANSIT
- OUT_FOR_DELIVERY
- DELIVERED
- CANCELLED

When a shipment is first created, its default status should be CREATED if the client does not supply a status.

5. Backend Requirements

Create a Spring Boot REST API using a conventional layered design:

- Controller layer
- Service layer
- Repository layer
- Entity/model
- Request/response DTOs
- Exception handling

Do not return the persistence entity directly from the API. Use DTOs and map between the DTO and entity.

Required REST endpoints

Method	Endpoint	Purpose	Successful response
POST	/api/shipments	Create a shipment	201 Created
GET	/api/shipments	List all shipments	200 OK
GET	/api/shipments?status=IN_TRANSIT	Filter by status	200 OK
GET	/api/shipments/{id}	Get one shipment	200 OK
PUT	/api/shipments/{id}	Update a shipment	200 OK
DELETE	/api/shipments/{id}	Delete a shipment	204 No Content

Backend behaviour

- Return 404 Not Found when a shipment ID does not exist.
- Return 409 Conflict when a duplicate tracking number is submitted.
- Return 400 Bad Request for invalid input or an invalid status.
- Validate request data on the backend. Frontend validation alone is not sufficient.
- Return errors in a consistent JSON structure.
- Do not expose Java stack traces, SQL statements, or internal exception details to the client.
- Configure CORS so the React application can call the API during local development.
- Use database migrations with Flyway or Liquibase. Flyway is recommended.

Suggested error response

```
{
  "timestamp": "2026-09-19T10:30:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Shipment with id 25 was not found",
  "path": "/api/shipments/25"
}
```

The exact format may differ, but it must be consistent and useful.

Example create request

```
{
  "trackingNumber": "SHP-1001",
  "senderName": "North Warehouse",
  "receiverName": "City Pharmacy",
  "origin": "Leeds",
  "destination": "London",
  "carrier": "QuickMove Logistics",
  "status": "CREATED",
  "expectedDeliveryDate": "2026-09-25"
}
```

Example successful response

```
{
  "id": 1,
  "trackingNumber": "SHP-1001",
  "senderName": "North Warehouse",
  "receiverName": "City Pharmacy",
  "origin": "Leeds",
  "destination": "London",
  "carrier": "QuickMove Logistics",
  "status": "CREATED",
  "expectedDeliveryDate": "2026-09-25",
  "createdAt": "2026-09-19T10:30:00Z",
  "updatedAt": "2026-09-19T10:30:00Z"
}
```

6. Frontend Requirements

Build a React single-page application with a clean, simple, responsive interface.

Required UI

A. Shipment List page

Display shipments in a table or responsive card list with:

- Tracking number
- Origin
- Destination
- Carrier
- Expected delivery date
- Status
- View, Edit, and Delete actions

The page must also include:

- An **Add Shipment** button
- A status filter with an **All** option
- Loading feedback while data is being fetched
- A friendly empty-state message
- A visible error message if the API request fails

B. Add/Edit Shipment form

- Include controls for every user-editable shipment field.
- Display field-level validation messages.
- Prevent submission when required fields are invalid.
- Load existing data when editing.
- Disable the submit button while saving to reduce duplicate submissions.
- Return to the list or detail page after a successful save.

C. Shipment Detail page or panel

Display all shipment information, including `createdAt` and `updatedAt`.

D. Delete confirmation

Ask the user to confirm before deleting a shipment.

Frontend behaviour

- Use React Router for navigation.
- Keep API access in a separate service/module instead of calling the API throughout UI components.
- Use environment configuration for the API base URL.
- Do not hard-code container hostnames into source code.
- Handle successful, loading, empty, validation, and error states.

7. Required Technology Stack

Backend

- Java 21
- Spring Boot 3.x
- Maven or Gradle; Maven is recommended
- Spring Web
- Spring Data JPA
- Bean Validation
- PostgreSQL driver
- Flyway or Liquibase
- JUnit 5
- Mockito

Frontend

- React 18 or later
- Vite
- React Router
- Axios or the browser Fetch API
- Vitest and React Testing Library
- Plain CSS, CSS Modules, or a small UI library

Runtime and source control

- PostgreSQL 16 or later
- Docker
- Docker Compose
- Git and a Git hosting service such as GitHub, GitLab, or Bitbucket

Do not introduce additional infrastructure unless it is clearly justified in the README.

8. Docker Requirements

The submitted repository must contain one Docker Compose configuration that starts:

1. The PostgreSQL database
2. The Spring Boot backend
3. The React frontend

The reviewer should be able to run the complete application with:

```
docker compose up --build
```

Recommended local ports:

Component	Port
Frontend	3000
Backend API	8080

Component	Port
PostgreSQL	5432

Requirements:

- Add a Dockerfile for the backend.
- Add a Dockerfile for the frontend.
- Use a named volume for PostgreSQL data.
- Use environment variables for database credentials and connection settings.
- Include health checks or reliable startup coordination so the backend does not fail merely because PostgreSQL is still starting.
- The frontend must reach the backend correctly when all services run through Docker Compose.
- Do not commit passwords used outside this local assessment environment.
- Do not require the reviewer to install Java, Node.js, or PostgreSQL when using Docker. Only Docker and Docker Compose should be required.

9. Testing Requirements

Backend tests

Write meaningful automated tests for business behaviour. At minimum include:

- Service test for successful shipment creation
- Service test for duplicate tracking number rejection
- Service test for shipment-not-found handling
- Controller/API test for valid input
- Controller/API test for invalid input
- Repository test for filtering by status, if a custom repository query is used

Use JUnit 5, Mockito, and Spring Boot testing support as appropriate.

Tests must not depend on the developer's manually running PostgreSQL instance. An isolated test database, test container, embedded alternative, or properly mocked repository is acceptable depending on the test level.

Frontend tests

At minimum include:

- Shipment list renders API data
- Form displays validation feedback for missing required input
- API failure displays a user-friendly error

Use Vitest and React Testing Library or an equivalent React testing setup.

Test commands

The README must give exact commands for running backend and frontend tests independently.

10. Suggested Repository Structure

```
logistics-shipment-tracker/  
+-- backend/  
|   +-- src/  
|   +-- Dockerfile  
|   +-- pom.xml  
+-- frontend/  
|   +-- src/  
|   +-- Dockerfile  
|   +-- package.json  
+-- compose.yaml  
+-- .env.example  
+-- .gitignore  
+-- README.md
```

The exact internal package structure is the candidate's decision, but it should be clear and maintainable.

11. Implementation Plan

Candidates may use their own approach. The following order is recommended:

1. Create the Git repository and top-level project structure.
2. Model the shipment entity, status enum, and database migration.
3. Implement repository and service-layer CRUD operations.
4. Add DTOs, validation, mapping, controller endpoints, and exception handling.
5. Test the REST API independently.
6. Create the React application, routes, API service, list, detail, and form UI.
7. Add frontend validation and handling for loading, empty, success, and failure states.
8. Write backend and frontend tests.
9. Add Dockerfiles and Docker Compose configuration.
10. Run the complete system from a clean environment.
11. Complete the README and make final Git commits.

12. README Requirements

The root `README.md` is part of the assessment. It must contain:

1. Project title and short description
2. Implemented features
3. Architecture/technology summary
4. Prerequisites
5. Exact command to start the complete application
6. Frontend URL and backend API URL
7. Commands to stop the application
8. Commands to run backend tests
9. Commands to run frontend tests

10. Any required environment variables
11. Example API requests, or a Postman collection/OpenAPI UI link
12. Assumptions and design decisions
13. Known limitations or incomplete work
14. Troubleshooting instructions for common startup issues

The reviewer must not need to contact the candidate to discover basic run instructions.

13. Git Submission Requirements

Submit a link to a repository that the interviewer can access.

The repository must:

- Contain all source code and documentation.
- Include a meaningful `.gitignore`.
- Exclude generated build output, dependency folders, IDE files, secrets, and database files.
- Have a reasonable sequence of descriptive commits rather than one unexplained final commit.
- Build from the committed files without depending on uncommitted local configuration.
- Include `.env.example` if environment variables are needed.
- State any use of AI coding assistance in the README, including what it helped with and how generated code was verified.

Do not commit real credentials or API keys.

14. Expected Execution Result

After cloning the repository, the interviewer should be able to run:

```
git clone <repository-url>
cd <repository-folder>
docker compose up --build
```

The expected outcome is:

1. PostgreSQL starts and becomes healthy.
2. The backend connects to PostgreSQL and applies database migrations.
3. The backend API starts successfully on port 8080.
4. The frontend starts successfully on port 3000.
5. Opening `http://localhost:3000` displays the Shipment List page.
6. A user can create a shipment and see it in the list.
7. Refreshing the browser retains the record because it is stored in PostgreSQL.
8. A user can view, edit, filter, and delete shipments.
9. Invalid data produces clear validation messages.
10. Backend and frontend test commands complete successfully.

The application should not require manual database table creation or source-code edits after cloning.

15. Acceptance Criteria

A submission is functionally complete when all of the following are true:

- ☐ All three services start with `docker compose up --build`.
- ☐ Shipment creation works from the React UI through the REST API to PostgreSQL.
- ☐ Shipment list and detail views show persisted data.
- ☐ Shipment update works and persists after refresh.
- ☐ Shipment deletion requires confirmation and removes the record.
- ☐ Filtering by status works.
- ☐ Duplicate tracking numbers are rejected.
- ☐ Invalid input is rejected by both the UI and backend.
- ☐ Missing shipment IDs return 404 with a clear JSON error.
- ☐ Required backend tests pass.
- ☐ Required frontend tests pass.
- ☐ The README provides complete setup and test instructions.
- ☐ No secrets or unnecessary generated files are committed.

16. Evaluation Rubric

Area	Weight	What the interviewer should assess
Functional correctness	25%	CRUD flow, status filter, persistence, validation, error scenarios
Backend quality	20%	REST design, layering, DTOs, exception handling, readable Java code
Frontend quality	15%	Component design, routing, forms, API integration, user feedback
Database design	10%	Appropriate types, uniqueness, migrations, persistence behaviour
Automated tests	15%	Meaningful coverage, clarity, isolation, passing test suite
Docker and execution	10%	Reliable one-command startup, configuration, clean container setup
Git and documentation	5%	Commit quality, README completeness, no secrets or build artifacts

Suggested decision guide

- **85-100:** Strong entry-level candidate; solution is reliable and candidate explains decisions clearly.
- **70-84:** Good candidate; core requirements work with minor gaps.
- **55-69:** Partial solution; assess fundamentals and learning ability during the live discussion.
- **Below 55:** Important requirements or core understanding are missing.

Do not score only by counting files or libraries. Give greater importance to working behaviour, clarity, and the candidate's ability to explain their own code.

17. Live Interview Discussion and Demonstration

Ask the candidate to use the submitted Git code and demonstrate:

1. Start the system using Docker Compose.
2. Create, view, update, filter, and delete a shipment.
3. Show one backend validation failure.
4. Run the automated tests.
5. Explain the request flow from React to Spring Boot to PostgreSQL.
6. Explain why DTOs and a service layer were used.
7. Explain how duplicate tracking numbers are prevented.
8. Explain how container-to-container networking works in the Compose setup.
9. Make one small live change, such as adding a `notes` field or a `RETURNED` status.
10. Identify one improvement they would make with more time.

Useful follow-up questions:

- What is the difference between POST and PUT?
- Why does create return 201, delete return 204, and missing data return 404?
- What happens if two requests create the same tracking number at nearly the same time?
- Why must backend validation remain even when the React form validates input?
- How would pagination be added if the table contained 100,000 shipments?
- How would this application be secured in production?
- What did you test, and what did you choose not to test?
- Which parts were written or suggested by an AI tool, and how did you verify them?

18. Optional Enhancements

These items are optional and must not compensate for missing required functionality:

- Pagination and sorting
- Search by tracking number
- OpenAPI/Swagger documentation
- Testcontainers for PostgreSQL integration tests
- Seed/demo data using a migration
- Improved accessibility
- A simple status summary showing counts by status
- CI workflow that builds and tests both applications

Candidates should clearly label optional work in the README.

19. Out of Scope

The following are deliberately excluded:

- Login, JWT, and role-based access
- Multiple microservices

- Kafka or other message brokers
- Cloud deployment
- Kubernetes
- Real carrier integrations
- Maps and live GPS tracking
- Email/SMS notifications
- File uploads
- Payment processing

Implementing these is not necessary for this assessment.

20. Submission Checklist for the Candidate

- ☐ I have implemented every required CRUD operation.
- ☐ I have implemented status filtering.
- ☐ I have validated input on both frontend and backend.
- ☐ I have added consistent API error handling.
- ☐ I have created database migrations.
- ☐ I have written and run backend tests.
- ☐ I have written and run frontend tests.
- ☐ I have run the entire application using Docker Compose.
- ☐ I have tested the instructions from a clean checkout if possible.
- ☐ I have not committed secrets, dependency directories, or build outputs.
- ☐ I have documented assumptions, limitations, and any AI assistance.
- ☐ I have shared an accessible Git repository URL.

Final Candidate Instruction

Prioritize a small, working, understandable solution over unnecessary complexity. Be prepared to run the application, explain the code, discuss design decisions, and make a small change during the interview.